

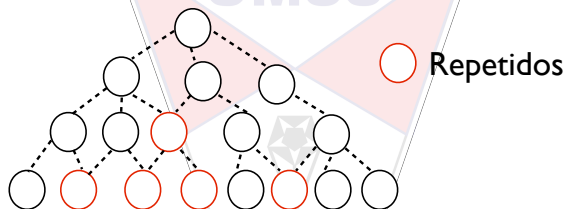


- ▶ programación dinámica

ámica UMSS
 n programación dinámica
 a estructura de una solución óptima
 te definir el valor de una solución óptima
 lor de una solución óptima con
 a solución óptima de la información

1. Caracterizar la estructura de una solución óptima.
2. Recursivamente definir el valor de una solución óptima.
3. Calcular el valor de una solución óptima con diseño bottom-up.
4. Construir una solución óptima de la información calculada (ya procesada).

Programación dinámica



“Programación dinámica”

Dividir un problema en subproblemas para resolverlo y evitar reprocesar la solución a un subsubproblema.

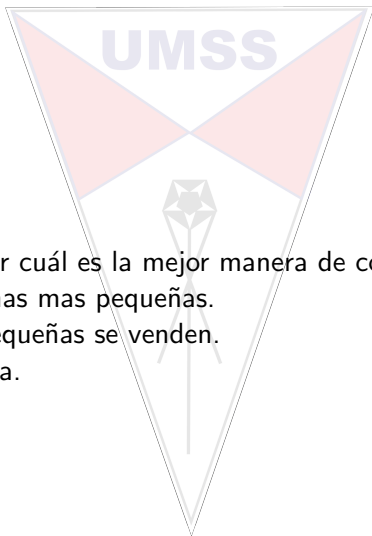
Para ello:

- ▶ resuelve cada subsubproblema sólo una vez y guarda su respuesta en una tabla
- ▶ anula el recalcularse la respuesta cada vez que el

subsubproblema es encontrado

Example

Consiste en decidir cuál es la mejor manera de cortar barras de
acero largas en unas mas pequeñas.
Las barras más pequeñas se venden.
Maximizar la venta.



[illegible]

- ▶ Barra de largo n .
- ▶ Tabla de longitudes de varillas que se venden.
- ▶ Los precios de cada una de ellas.

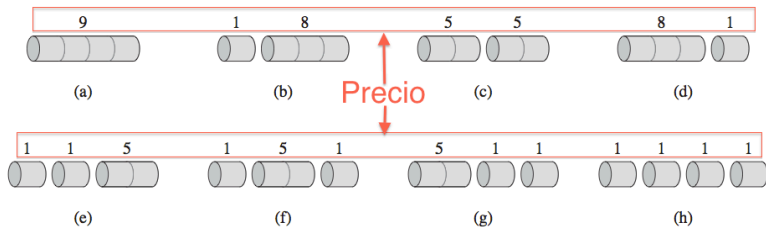
largo: i	1	2	3	4	5	6	7	8	9	10
precio: p_i	1	5	8	9	10	17	17	20	24	30

Cuál es la mejor manera de vender la barra de tamaño n ?

Corte de barras

largo: i	1	2	3	4	5	6	7	8	9	10
precio: p_i	1	5	8	9	10	17	17	20	24	30

Por ejemplo si se tiene una barra de tamaño 4, los posibles cortes que se pueden hacer son:



La respuesta es corte: 2, 2

Básicamente es encontrar todas las sumas que me den n , es decir:

Country	1950	1960	1970	1980	1990	2000	2010	2020	2030	2040	2050
Japan	7	8	10	12	14	16	18	20	22	24	26
Germany	10	11	12	13	14	15	16	17	18	19	20
France	11	12	13	14	15	16	17	18	19	20	21
Italy	12	13	14	15	16	17	18	19	20	21	22
Spain	13	14	15	16	17	18	19	20	21	22	23
United Kingdom	14	15	16	17	18	19	20	21	22	23	24
United States	15	16	17	18	19	20	21	22	23	24	25
Canada	16	17	18	19	20	21	22	23	24	25	26
Sweden	17	18	19	20	21	22	23	24	25	26	27
Netherlands	18	19	20	21	22	23	24	25	26	27	28
Belgium	19	20	21	22	23	24	25	26	27	28	29
Australia	20	21	22	23	24	25	26	27	28	29	30
South Korea	21	22	23	24	25	26	27	28	29	30	31
India	22	23	24	25	26	27	28	29	30	31	32
China	23	24	25	26	27	28	29	30	31	32	33
Brazil	24	25	26	27	28	29	30	31	32	33	34
Argentina	25	26	27	28	29	30	31	32	33	34	35
South Africa	26	27	28	29	30	31	32	33	34	35	36
Indonesia	27	28	29	30	31	32	33	34	35	36	37
Philippines	28	29	30	31	32	33	34	35	36	37	38
Thailand	29	30	31	32	33	34	35	36	37	38	39
Malaysia	30	31	32	33	34	35	36	37	38	39	40
Singapore	31	32	33	34	35	36	37	38	39	40	41
Hong Kong	32	33	34	35	36	37	38	39	40	41	42
Taiwan	33	34	35	36	37	38	39	40	41	42	43
South Korea	34	35	36	37	38	39	40	41	42	43	44
Japan	35	36	37	38	39	40	41	42	43	44	45
Germany	36	37	38	39	40	41	42	43	44	45	46
France	37	38	39	40	41	42	43	44	45	46	47
Italy	38	39	40	41	42	43	44	45	46	47	48
Spain	39	40	41	42	43	44	45	46	47	48	49
United Kingdom	40	41	42	43	44	45	46	47	48	49	50
United States	41	42	43	44	45	46	47	48	49	50	51
Canada	42	43	44	45	46	47	48	49	50	51	52
Sweden	43	44	45	46	47	48	49	50	51	52	53
Netherlands	44	45	46	47	48	49	50	51	52	53	54
Belgium	45	46	47	48	49	50	51	52	53	54	55
Australia	46	47	48	49	50	51	52	53	54		

1

— — — — —

Una solución recursiva es, con facción top down:

$$\text{CUT-ROD}(p, n)$$
$$1 \quad \text{if } n == 0$$

```
2     return 0
```

3 $q = -\infty$

4 **for** $i = 1$ **to** n

$$5 \quad q = \max(q, p[i] + \text{CUT-ROD}(p, n - i))$$
6 return q 

proceso para $n = 4$:

La ejecución del proceso para $n = 4$:



Una solución mejor top down es recordando los anteriores calculos ("memos" a lo cual se llama memoización):

MEMOIZED-CUT-ROD(p, n)

```
1  let  $r[0..n]$  be a new array
```

2 **for** $i = 0$ **to** n

$$3 \quad r[i] = -\infty$$

```

4  return MEMOIZED-CUT-ROD-AUX( $p, n, r$ )

```



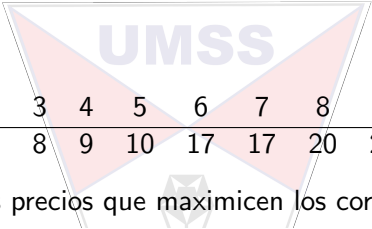
```

1  if  $r[n] \geq 0$ 
2      return  $r[n]$ 
3  if  $n == 0$ 
4       $q = 0$ 
5  else  $q = -\infty$ 
6      for  $i = 1$  to  $n$ 
7           $q = \max(q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r))$ 
8   $r[n] = q$ 
9  return  $q$ 

```



Corte de barras



largo: i	1	2	3	4	5	6	7	8	9	10
precio: p_i	1	5	8	9	10	17	17	20	24	30

Encontraremos los precios que maximicen los cortes “vendibles”

$r_1 = 1$ from solution 1 = 1 (no cuts),

$r_2 = 5$ from solution 2 = 2 (no cuts),

$r_3 = 8$ from solution 3 = 3 (no cuts),

$r_4 = 10$ from solution 4 = 2 + 2,

$r_5 = 13$ from solution 5 = 2 + 3,

$r_6 = 17$ from solution 6 = 6 (no cuts),

$r_7 = 18$ from solution 7 = 1 + 6 or 7 = 2 + 2 + 3,

$r_8 = 22$ from solution 8 = 2 + 6,

$r_9 = 25$ from solution 9 = 3 + 6,

$r_{10} = 30$ from solution 10 = 10 (no cuts)

Mejor que no
cortar

Corte de barras

Cómo se encuentra esto?



$r_1 = 1$ from solution 1 = 1 (no cuts) ,
 $r_2 = 5$ from solution 2 = 2 (no cuts) ,
 $r_3 = 8$ from solution 3 = 3 (no cuts) ,
 $r_4 = 10$ from solution 4 = 2 + 2 ,
 $r_5 = 13$ from solution 5 = 2 + 3 ,
 $r_6 = 17$ from solution 6 = 6 (no cuts) ,
 $r_7 = 18$ from solution 7 = 1 + 6 or 7 = 2 + 2 + 3 ,
 $r_8 = 22$ from solution 8 = 2 + 6 ,
 $r_9 = 25$ from solution 9 = 3 + 6 ,
 $r_{10} = 30$ from solution 10 = 10 (no cuts) .

$$\begin{aligned}
 &= \max(p_7, r_1 + r_6, r_2 + r_5, r_3 + r_4) \\
 &= \max(17, 1 + 17, 5 + 13, 8 + 10) \\
 &= \max(17, 18, 18, 18) \\
 &= 18
 \end{aligned}$$

Que responde a la siguiente abstracción:

$$r_n = \max(p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1)$$

Corte de barras

Formalizando esto es:

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

Probemos para $n = 7$:

i	p_i		r_{n-i}		
1:	1	+	17	=	18
2:	5	+	13	=	18
3:	8	+	10	=	18
4:	9	+	8	=	17
5:	10	+	5	=	15
6:	17	+	1	=	18
7:	17	+	0	=	17

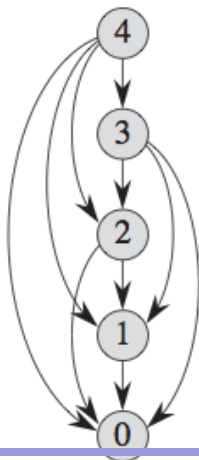
largo: i	1	2	3	4	5	6	7	8	9	10
precio: p_i	1	5	8	9	10	17	17	20	24	30
precioMax: r_i	1	5	8	10	13	17	?	-	-	-

Corte de barras

Una solución más eficiente bottom up es :

BOTTOM-UP-CUT-ROD(p, n)

```
1  let  $r[0..n]$  be a new array
2   $r[0] = 0$ 
3  for  $j = 1$  to  $n$ 
4       $q = -\infty$ 
5      for  $i = 1$  to  $j$ 
6           $q = \max(q, p[i] + r[j - i])$ 
7       $r[j] = q$ 
8  return  $r[n]$ 
```

Corte de barras

Algoritmo que además calcula el mejor primer corte de la barra

EXTENDED-BOTTOM-UP-CUT-ROD(p, n)

```

1  let  $r[0..n]$  and  $s[0..n]$  be new arrays
2   $r[0] = 0$ 
3  for  $j = 1$  to  $n$ 
4       $q = -\infty$ 
5      for  $i = 1$  to  $j$ 
6          if  $q < p[i] + r[j - i]$ 
7               $q = p[i] + r[j - i]$ 
8               $s[j] = i$ 
9       $r[j] = q$ 
10 return  $r$  and  $s$ 

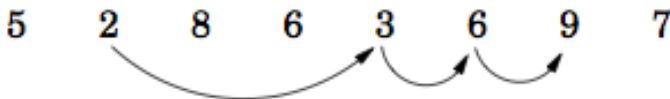
```

fib(n) = fib(n-1) + fib(n-2)
fib(1) = 1
fib(2) = 1

- ▶ considera la solución recursiva (top down)
- ▶ considera la solución recursiva (con memoización)
- ▶ considera la solución con programación dinámica (bottom up)

Example

Dada una secuencia de números enteros no ordenados se pide encontrar la subsecuencia incremental mas larga, consecutiva o no de la secuencia ¹ original.



¹Pueden no estar consecutivos pero en un orden topológico de visita deberían estar en una ubicación creciente

El algoritmo es simple:

```

for  $j = 1, 2, \dots, n$ :
     $L(j) = 1 + \max\{L(i) : (i, j) \in E\}$ 
return  $\max_j L(j)$ 

```

Así como es el elemento más importante de la programación dinámica, es necesario recordar que:

“En programación dinámica la clave es dividir en subproblemas de manera que sus resultados sean informacion vital a recordar y acarrear hacia adelante en la búsqueda de la solución”

```
for(int i = 0; i < n; i++){
    sec = in.nextInt();
    suma += sec;
    if(suma > resp)
        resp = suma;
    if(suma < 0)
        suma = 0;
}
System.out.println(resp);
```


Máxima suma 2D

Es el mismo problema pero ahora se tiene la colección de números organizada como una matriz. Entonces, se quiere saber cual es la submatriz de maxima suma.

0	-2	-7	0	==>	0	-2	-7	0
9	2	-6	2	==>	9	2	-6	2
-4	1	-4	1	==>	-4	1	-4	1
-1	8	0	-2	==>	-1	8	0	-2

16 de 1x1

12 de 2x1

8 de 3x1

4 de 4x1

12 de 1x2

9 de 2x2

6 de 3x2

3 de 4x2

8 de 1x3

6 de 2x3

4 de 3x3

2 de 4x3

4 de 1x4

3 de 2x4

2 de 3x4

1 de 4x4

Máxima suma 2D

Para ello, se realiza la suma total de los elementos hasta ahora visitados cuidando de formar matrices.

Por ejemplo el **-4**, es el resultado de sumar las siguientes celdas:

$$\begin{array}{ccccccc}
 0 & -2 & -7 & 0 & ==> & 0 & -2 & -9 & -9 \\
 9 & \boxed{2} & \boxed{-6} & 2 & ==> & 9 & 9 & \boxed{-4} & -2 \\
 -4 & \boxed{1} & -4 & 1 & ==> & 5 & 6 & -11 & -8 \\
 -1 & \boxed{8} & \boxed{0} & -2 & ==> & 4 & 13 & -4 & -3
 \end{array}$$

Cada celda suma, se consigue así:

$$suma[i,j] = sec[i,j] + suma[i-1, j] + suma[i,j-1] - suma[i-1,j-1]$$

A excepción de los valores frontera que se encuentran a la izquierda y arriba de la matriz.

$$suma[0, j] = sec[0, j] + suma[0, j-1], j > 0$$

$$suma[i, 0] = sec[i, 0] + suma[i-1, 0], i > 0$$

$$suma[0, 0] = sec[0, 0]$$

```
for(int i =0; i < m; i++){
    for(int j = 0; j < n; j++){
        sec[i][j] = in.nextInt();
        suma[i][j] = sec[i][j];
        if(i > 0)
            suma[i][j] += suma[i-1][j];
        if(j > 0)
            suma[i][j] += suma[i][j-1];
        if(i > 0 && j > 0)
            suma[i][j] -= suma[i-1][j-1];
    }
}
```

Cuidando los casos especiales cuando los valores estan en frontera.

Esta solución toma $O(n^4)$, el costo mayor esta en la formación de las submatrices, ya que se requiere generar los cuatro subíndices:

 i, j, k, l

La solución sin DP, cuesta $O(n^6)$.

El problema consiste en cargar una mochila con ítemes de diferentes tamaños de tal manera de optimizar la carga. Hay variantes de este problema, que consideran diferentes criterios de optimización, y la estrategia de ir cargando la mochila 0-1.

knapsack sin repetición: hace referencia al hecho de tomar o no un ítem para ponerlo a la mochila (0 no tomar, 1 tomar). *knapsack 0-1*

Para aplicar programación dinámica hay que detectar la posibilidad de resolver un problema de igual naturaleza y ver que este resultado sea reutilizado para resolver el problema original que es mayor. Probablemente, es correcto pensar que si se puede llenar *mochilas de menor capacidad con optimalidad* esto podría ayudar a llenar la mochila original.

Knapsack con repetición - PD

$k(w)$ es una mochila de capacidad w y que es optima.

Si se asume que en la mochila está el elemento i -esimo, entonces se puede decir que:

$$k(w) = k(w - w_i) + v_i$$

Donde w hace referencia al peso, y v hace referencia al valor del item.

De hecho $k(w - w_i)$ es optimo.

Por otro lado si no se conoce cuál es el i -esimo elemento que mantiene la optimalidad, entonces, se debe buscar el i -esimo elemento entre todas las opciones.

Por lo que:

$$k(w) = \max_{i: w_i \leq w} \{k(w - w_i) + v_i\}$$

Competición - PD

$$K(0) = 0$$

for $w = 1$ to W :

$$K(w) = \max\{K(w - w_i) + v_i : w_i \leq w\}$$

```
return  $K(W)$ 
```



Por ejemplo: si se quiere maximizar una mochila de capacidad $W = 10$, con ítemes:

Item	1	2	3	4
Peso	6	3	4	2
Valor	30	14	16	9

Se construye un memo de los valores para mochilas de capacidades 0 a W :

[illegible]

Knapsack con repetición - PD

Item	1	2	3	4
Peso	6	3	4	2
Valor	30	14	16	9

$\text{MEMO}[1] = 0$ porque no hay ningún ítem que pueda ser ingresado en una mochila de capacidad 1.

MEMO =

0	1	2	3	4	5	6	7	8	9	10
0	0									

Knapsack con repetición - PD

Item	1	2	3	4
Peso	6	3	4	2
Valor	30	14	16	9

$$\text{MEMO}[2] = \max(\text{MEMO}[2 - 2] + 9) = \max(0 + 9) = 9$$

MEMO =

0	1	2	3	4	5	6	7	8	9	10
0	0	9								

Knapsack con repetición - PD

Item	1	2	3	4
Peso	6	3	4	2
Valor	30	14	16	9

$$\text{MEMO}[3] = \max(\text{MEMO}[3 - 3] + 14, \text{MEMO}[3 - 2] + 9) = \max(14, 9) = 14$$

MEMO =

0	1	2	3	4	5	6	7	8	9	10
0	0	9	14							

Knapsack con repetición - PD

Item	1	2	3	4
Peso	6	3	4	2
Valor	30	14	16	9

$$\begin{aligned} \text{MEMO}[4] &= \max(\text{MEMO}[4 - 3] + 14, \text{MEMO}[4 - 4] + 16, \\ &\text{MEMO}[4 - 2] + 9) = \max(14, 16, 18) = 18 \end{aligned}$$

MEMO =

0	1	2	3	4	5	6	7	8	9	10
0	0	9	14	18						

Knapsack con repetición - PD

Item	1	2	3	4
Peso	6	3	4	2
Valor	30	14	16	9

$$\text{MEMO}[5] = \max(\text{MEMO}[5 - 3] + 14, \text{MEMO}[5 - 4] + 16, \text{MEMO}[5 - 2] + 9) = \max(23, 16, 23) = 23$$

MEMO =

0	1	2	3	4	5	6	7	8	9	10
0	0	9	14	18	23					

Knapsack con repetición - PD

Item	1	2	3	4
Peso	6	3	4	2
Valor	30	14	16	9

$$\begin{aligned} \text{MEMO}[6] &= \max(\text{MEMO}[6 - 6] + 30, \text{MEMO}[6 - 3] + 14, \\ &\text{MEMO}[6 - 4] + 16, \text{MEMO}[6 - 2] + 9) = \max(30, 28, 25, 27) = \\ &30 \end{aligned}$$

MEMO =

0	1	2	3	4	5	6	7	8	9	10
0	0	9	14	18	23	30				

Knapsack con repetición - PD

Item	1	2	3	4
Peso	6	3	4	2
Valor	30	14	16	9

$$\begin{aligned} \text{MEMO}[7] &= \max(\text{MEMO}[7 - 6] + 30, \text{MEMO}[7 - 3] + 14, \\ &\text{MEMO}[7 - 4] + 16, \text{MEMO}[7 - 2] + 9) = \max(30, 32, 30, 32) = \\ &32 \end{aligned}$$

MEMO =

0	1	2	3	4	5	6	7	8	9	10
0	0	9	14	18	23	30	32			

Knapsack con repetición - PD

Item	1	2	3	4
Peso	6	3	4	2
Valor	30	14	16	9

$$\begin{aligned} \text{MEMO}[8] &= \max(\text{MEMO}[8 - 6] + 30, \text{MEMO}[8 - 3] + 14, \\ &\text{MEMO}[8 - 4] + 16, \text{MEMO}[8 - 2] + 9) = \max(39, 37, 34, 39) = 39 \end{aligned}$$

MEMO =

0	1	2	3	4	5	6	7	8	9	10
0	0	9	14	18	23	30	32	39		

Knapsack con repetición - PD

Item	1	2	3	4
Peso	6	3	4	2
Valor	30	14	16	9

$\text{MEMO}[9] = \max(\text{MEMO}[9 - 6] + 30, \text{MEMO}[9 - 3] + 14,$
 $\text{MEMO}[9 - 4] + 16, \text{MEMO}[9 - 2] + 9) = \max(44, 44, 39, 41) =$
 44

MEMO =

0	1	2	3	4	5	6	7	8	9	10
0	0	9	14	18	23	30	32	39	44	

Knapsack con repetición - PD

Item	1	2	3	4
Peso	6	3	4	2
Valor	30	14	16	9

$$\begin{aligned} \text{MEMO}[10] &= \max(\text{MEMO}[10 - 6] + 30, \text{MEMO}[10 - 3] + 14, \\ &\text{MEMO}[10 - 4] + 16, \text{MEMO}[10 - 2] + 9) = \max(48, 46, 46, 48) \\ &= 48 \end{aligned}$$

MEMO =

0	1	2	3	4	5	6	7	8	9	10
0	0	9	14	18	23	30	32	39	44	48

$k(w, j)$ es el máximo valor alcanzado para una mochila de capacidad w con j ítemes, desde 1 a j .

(c) $\frac{1}{\sqrt{2}}$


```

Initialize all  $K(0, j) = 0$  and all  $K(w, 0) = 0$ 
for  $j = 1$  to  $n$ :
    for  $w = 1$  to  $W$ :
        if  $w_j > w$ :  $K(w, j) = K(w, j - 1)$ 
        else:  $K(w, j) = \max\{K(w, j - 1), K(w - w_j, j - 1) + v_j\}$ 
return  $K(W, n)$ 

```

Una mochila de capacidad $W = 20$, cargar con ítemes que maximicen el valor de la carga.

Item	1	2	3	4	5	6
Peso	2	4	6	3	4	8
Valor	10	5	20	2	6	1

En este caso se usa un MEMO de dos dimensiones que calcule mochilas de menor capacidad con los elementos tomados solo una vez, ya que aqui se necesita saber la mejor mochila con o sin el item.

Knapsack sin repetición - PD

Para $W = 20$ e Items =

Item	1	2	3	4	5	6
Peso	2	4	6	3	4	8
Valor	10	5	20	2	6	1

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0						
2	0						
3	0						
4	0						
5	0						
6	0						
7	0						
8	0						
9	0						
10	0						
11	0						
12	0						
13	0						
14	0						
15	0						
16	0						
17	0						
18	0						
19	0						
20	0						

.....

Knapsack sin repetición - PD

Para $W = 20$ e Items =

Item	1	2	3	4	5	6
Peso	2	4	6	3	4	8
Valor	10	5	20	2	6	1

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	0					
2	0	10					
3	0	10					
4	0	10					
5	0	10					
6	0	10					
7	0	10					
8	0	10					
9	0	10					
10	0	10					
11	0	10					
12	0	10					
13	0	10					
14	0	10					
15	0	10					
16	0	10					
17	0	10					
18	0	10					
19	0	10					
20	0	10					

w

$1; w_1 > w : M[1][1] = M[1][0] = 0$
 $2; w_1 < w : M[2][1] = \max(M[2][0], M[0][0] + 10)$
 $3; w_1 < w : M[3][1] = \max(M[3][0], M[1][0] + 10)$
 $4; w_1 < w : M[4][1] = \max(M[4][0], M[2][0] + 10)$
 $5; w_1 < w : M[5][1] = \max(M[5][0], M[3][0] + 10)$
 $6; w_1 < w : M[6][1] = \max(M[6][0], M[4][0] + 10)$
 $7; w_1 < w : M[7][1] = \max(M[7][0], M[5][0] + 10)$
 $8; w_1 < w : M[8][1] = \max(M[8][0], M[6][0] + 10)$
 $9; w_1 < w : M[9][1] = \max(M[9][0], M[7][0] + 10)$
 $10; w_1 < w : M[10][1] = \max(M[10][0], M[8][0] + 10)$
 $11; w_1 < w : M[11][1] = \max(M[11][0], M[9][0] + 10)$
 $12; w_1 < w : M[12][1] = \max(M[12][0], M[10][0] + 10)$
 $13; w_1 < w : M[13][1] = \max(M[13][0], M[11][0] + 10)$
 $14; w_1 < w : M[14][1] = \max(M[14][0], M[12][0] + 10)$
 $15; w_1 < w : M[15][1] = \max(M[15][0], M[13][0] + 10)$
 $16; w_1 < w : M[16][1] = \max(M[16][0], M[14][0] + 10)$
 $17; w_1 < w : M[17][1] = \max(M[17][0], M[15][0] + 10)$
 $18; w_1 < w : M[18][1] = \max(M[18][0], M[16][0] + 10)$
 $19; w_1 < w : M[19][1] = \max(M[19][0], M[17][0] + 10)$
 $20; w_1 < w : M[20][1] = \max(M[20][0], M[18][0] + 10)$

Knapsack sin repetición - PD

Para $W = 20$ e Items =

Item	1	2	3	4	5	6
Peso	2	4	6	3	4	8
Valor	10	5	20	2	6	1

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	0	0	0			
2	0	10	10	10			
3	0	10	10	10			
4	0	10	10	10			
5	0	10	10	10			
6	0	10	15				
7	0	10	15				
8	0	10	15				
9	0	10	15				
10	0	10	15				
11	0	10	15				
12	0	10	15				
13	0	10	15				
14	0	10	15				
15	0	10	15				
16	0	10	15				
17	0	10	15				
18	0	10	15				
19	0	10	15				
20	0	10	15				

w

- $1; w_2 > w : M[1][2] = M[1][1]$
 $2; w_2 > w : M[2][2] = M[2][1]$
 $3; w_2 > w : M[3][2] = M[3][1]$
 $4; w_2 \leq w : M[4][2] = \max(M[4][1], M[0][1] + 5)$
 $5; w_2 \leq w : M[5][2] = \max(M[5][1], M[1][1] + 5)$
 $6; w_2 \leq w : M[6][2] = \max(M[6][1], M[2][1] + 5)$
 $7; w_2 \leq w : M[7][2] = \max(M[7][1], M[3][1] + 5)$
 $8; w_2 \leq w : M[8][2] = \max(M[8][1], M[4][1] + 5)$
 $9; w_2 \leq w : M[9][2] = \max(M[9][1], M[5][1] + 5)$
 $10; w_2 \leq w : M[10][2] = \max(M[10][1], M[6][1] + 5)$
 $11; w_2 \leq w : M[11][2] = \max(M[11][1], M[7][1] + 5)$
 $12; w_2 \leq w : M[12][2] = \max(M[12][1], M[8][1] + 5)$
 $13; w_2 \leq w : M[13][2] = \max(M[13][1], M[9][1] + 5)$
 $14; w_2 \leq w : M[14][2] = \max(M[14][1], M[10][1] + 5)$
 $15; w_2 \leq w : M[15][2] = \max(M[15][1], M[11][1] + 5)$
 $16; w_2 \leq w : M[16][2] = \max(M[16][1], M[12][1] + 5)$
 $17; w_2 \leq w : M[17][2] = \max(M[17][1], M[13][1] + 5)$
 $18; w_2 \leq w : M[18][2] = \max(M[18][1], M[14][1] + 5)$
 $19; w_2 \leq w : M[19][2] = \max(M[19][1], M[15][1] + 5)$
 $20; w_2 \leq w : M[20][2] = \max(M[20][1], M[16][1] + 5)$

Ejercicio UVA 10688 - The Poor Giant

Problema: encontrar una manzana dulce entre amargas y ácidas.

Condiciones: las amargas son ligeramente mas livianas las agrias son ligeramente mas pesadas; la i -esima manzana pesa $k + i$.

Resultado: lo que se busca es la suma de los pesos de las manzanas que debe comerse para encontrar la manzana dulce.

En al descripcion deberia ser $1+4+4+4 = 13$. Como se consigue?

- si la 2 es dulce entonces como un peso de 2

9 1

- si la 1 es dulce entonces come un peso de 1
- si la 1 no es dulce, entonces la dulce esta adelante
- si la 3 es dulce, comio $1+3$.
- si la 3 si es amarga, entonces la dulce es la 4, comio $1+3$.
- si la 3 si es agria, entonces la dulce es la 2, comio $1+3$.

peso total es $1 + 4 + 4 + 4 = 13$

Es mejor opción

1	2	3	4	SUMA
1	4	4	4	13
2	2	5	5	14
4	4	3	3	14
6	6	6	4	22

La mínima suma es $13 = 1 + 4 + 4 + 4$.

1	2	3	4	5	SUMA
1	4	4	8	8	25
2	2	6	6	6	22
4	4	3	7	7	25
7	7	10	10	5	39

La mínima suma es $22 = 2 + 2 + 6 + 6 + 6$.

Ejercicio UVA 10688 - The Poor Giant

Para el ejemplo: $n = 10$, $k = 20$:

	21	22	23	24	25	26	27	28	29	30	SUMA
1	21	60	60	84	84	37	65	65	94	94	664
2	22	22	72	72	72	48	76	76	105	105	670
3	44	44	23	75	75	75	50	79	79	79	623
4	46	46	46	24	76	76	51	80	80	80	605
5	47	47	70	70	25	79	79	53	82	82	634
6	70	70	49	73	73	26	54	54	83	83	635
7	71	71	50	75	75	75	27	56	56	56	612
8	74	74	74	52	78	78	78	28	57	57	650
9	75	75	75	53	79	79	106	106	29	29	706
10	77	77	100	100	55	82	82	110	110	30	823
